PHASE 3 Redis

We will implement Redis in this order:

☑ Step 1 → Add Redis container
☑ Step 2 → Connect Spring Boot to Redis
☑ Step 3 → Enable caching
☑ Step 4 → Cache Product APIs
☑ Step 5 → Handle Cache Eviction
☑ Step 6 → Verify working

☑ STEP 1: Add Redis in docker-compose

Open your:

docker-compose.yml

Add Redis service.

```
redis:
  image: redis:7
  container_name: redis
  ports:
    - "6379:6379"
  restart: always
```

Your compose will look like:

```
version: "3.8"

services:

  postgres-db:
    image: postgres:17
    container_name: postgres-db
    restart: always
    environment:
      POSTGRES_DB: advann_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: root
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
  redis:
    image: redis:7
    container_name: redis
    restart: always
```

```
  ports:
    - "6379:6379"

eureka-server:
  build: ./eureka-service-registry
  container_name: eureka-server
  restart: always
  ports:
    - "8761:8761"
  depends_on:
    - postgres-db

api-gateway:
  build: ./api-gateway
  container_name: api-gateway
  restart: always
  ports:
    - "8080:8080"
  depends_on:
    - eureka-server

user-service:
  build: ./user-service
  container_name: user-service
  restart: always
  ports:
    - "8083:8083"
  depends_on:
    - eureka-server
    - postgres-db

product-service:
  build: ./product-service
  container_name: product-service
  restart: always
  ports:
    - "8081:8081"
  depends_on:
    - eureka-server
    - postgres-db
  environment:
    AWS_ACCESS_KEY: ${AWS_ACCESS_KEY}
    AWS_SECRET_KEY: ${AWS_SECRET_KEY}
    AWS_REGION: ${AWS_REGION}
    AWS_BUCKET_NAME: ${AWS_BUCKET_NAME}

cart-service:
```

```
    build: ./cart-service
    container_name: cart-service
    restart: always
    ports:
      - "8082:8082"
    depends_on:
      - eureka-server
      - postgres-db

  order-service:
    build: ./order-service
    container_name: order-service
    restart: always
    ports:
      - "8084:8084"
    depends_on:
      - eureka-server
      - postgres-db

  payment-service:
    build: ./payment-service
    container_name: payment-service
    restart: always
    ports:
      - "8085:8085"
    depends_on:
      - eureka-server
      - postgres-db
    environment:
      RAZORPAY_KEY_ID: ${RAZORPAY_KEY_ID}
      RAZORPAY_KEY_SECRET: ${RAZORPAY_KEY_SECRET}

volumes:
  postgres_data:
```

☑ Run Containers

Stop everything:

```
docker-compose down
```

Start again:

```
docker-compose up -d
```

Check:

```
docker ps
```

You should see:

redis

☑ Redis server running.

☑ STEP 2: Add Redis Dependency

Go to **product-service**

Maven dependency

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

Reload Maven.

☑ STEP 3: Redis Configuration

Open:

```
product-service
→ application.yml
```

Add:

```
spring:
  data:
    redis:
      host: redis
      port: 6379
```

⚠ Important

Inside Docker use:

host: redis

NOT localhost.

☑ STEP 4: Enable Caching

In your **main class**

```
@SpringBootApplication
@EnableCaching
public class ProductServiceApplication {
}
```

Now Spring cache system is enabled.

☑ STEP 5: Cache Product Fetch API

Go to:

ProductService

Suppose you have:

public Product getProductById(Long id)

Modify like this:

```
@Cacheable(value = "products", key = "#id")
public Product getProductById(Long id) {

    return productRepository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("Product not found"));
}
```

☑ What Happens Now

First request:

API → DB → Redis stored

Second request:

API → Redis → FAST RESPONSE

DB not called.

So your code will be

```
@Override

@Cacheable(value = "products", key = "#id")

public ProductResponseDto getProductById(Long id) {


    log.info("Fetching product by id from DB: {}", id);


    Product product = productRepository.findById(id)

            .orElseThrow(() -> {

                log.warn("Product not found with id: {}", id);

                return new ResourceNotFoundException(

                        "Product not found with id: " + id);

            });
```

```
    ProductResponseDto dto =
            modelMapper.map(product, ProductResponseDto.class);

    dto.setImageUrl(buildImageUrl(product.getImagePath()));

    if (product.getCategory() != null) {
        dto.setCategoryId(product.getCategory().getId());
        dto.setCategoryName(product.getCategory().getName());
    }

    if (product.getSubCategory() != null) {
        dto.setSubCategoryId(product.getSubCategory().getId());
        dto.setSubCategoryName(product.getSubCategory().getName());
    }

    return dto;
}
```

☑ STEP 6: Cache Eviction (VERY IMPORTANT)

Whenever product updates, cache must refresh.

Example:

```
@CacheEvict(value = "products", key = "#id")
public Product updateProduct(Long id, Product dto) {

    Product product =
        productRepository.findById(id).orElseThrow();

    product.setName(dto.getName());
    product.setPrice(dto.getPrice());

    return productRepository.save(product);
}
```

Otherwise stale data appears.

Your code will look like :

```
@Override
@CacheEvict(value = "products", key = "#id")
public ProductResponseDto updateProduct(Long id,
                                        ProductRequestDto productRequestDto) {

    log.info("Updating product with id: {}", id);

    Product existing = productRepository.findById(id)
            .orElseThrow(() -> {
                log.warn("Cannot update. Product not found with id: {}", id);
                return new ResourceNotFoundException(
                        "Product not found with id: " + id);
            });

    existing.setName(productRequestDto.getName());
    existing.setPrice(productRequestDto.getPrice());
    existing.setStock(productRequestDto.getStock());

    Category category = categoryRepository
            .findById(productRequestDto.getCategoryId())
            .orElseThrow(() ->
                    new ResourceNotFoundException(
                            "Category not found with id: "
                                    + productRequestDto.getCategoryId()));

    SubCategory subCategory = subCategoryRepository
            .findById(productRequestDto.getSubCategoryId())
            .orElseThrow(() ->
                    new ResourceNotFoundException(
                            "SubCategory not found with id: "
                                    + productRequestDto.getSubCategoryId()));
```

```
    if (!subCategory.getCategory().getId()
            .equals(category.getId())) {
        throw new ResourceNotFoundException(
                "SubCategory does not belong to Category");
    }

    existing.setCategory(category);
    existing.setSubCategory(subCategory);

    Product updated = productRepository.save(existing);

    ProductResponseDto responseDto =
            modelMapper.map(updated, ProductResponseDto.class);

    responseDto.setCategoryId(category.getId());
    responseDto.setCategoryName(category.getName());

    responseDto.setSubCategoryId(subCategory.getId());
    responseDto.setSubCategoryName(subCategory.getName());

    responseDto.setImageUrl(
            buildImageUrl(updated.getImagePath()));

    return responseDto;
}
```

☑ **STEP 7: After updating in ProductServiceImpl file we will get to dto classes (Otherwise serialize error will come)**

```
package com.advann.product_service.dto;

import lombok.*;
```

```java
import java.io.Serializable;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class CategoryResponseDto implements Serializable {
    private static final long serialVersionUID = 1L;
    private Long id;
    private String name;
}
```

```java
package com.advann.product_service.dto;

import lombok.*;

import java.io.Serializable;

@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class ProductImageResponseDto implements Serializable {
    private static final long serialVersionUID = 1L;

    private Long id;

    private String imagePath;
    private String imageUrl;

    private String thumbnailPath;
    private String thumbnailUrl;
}
```

```java
package com.advann.product_service.dto;

import lombok.*;
import java.io.Serializable; // 1. Add this import
import java.math.BigDecimal;

@Data
@NoArgsConstructor
```

```
@AllArgsConstructor
@Builder
public class ProductResponseDto implements Serializable {
    private static final long serialVersionUID = 1L;

    private Long id;
    private String name;
    private BigDecimal price;
    private Integer stock;
    private String imagePath;
    private String imageUrl;
    private Long categoryId;
    private String categoryName;
    private Long subCategoryId;
    private String subCategoryName;
}
```

```
package com.advann.product_service.dto;

import lombok.Builder;
import lombok.Data;

import java.io.Serializable;

@Data
@Builder
public class SubCategoryResponseDto implements Serializable {
    private static final long serialVersionUID = 1L;
    private Long id;
    private String name;
    private Long categoryId;
    private String categoryName;
}
```

☑ STEP 8: After updating in GlobalExceptionHandler

```
@ExceptionHandler(Exception.class)
public ResponseEntity<ApiResponse<Object>> handleGenericException(Exception ex) {
    log.error("Internal Server Error occurred: ", ex);
    String errorMessage = "Something went wrong: " + ex.getMessage();
    if (ex.getCause() != null) {
        errorMessage += " | Cause: " + ex.getCause().getClass().getSimpleName();
    }

    ApiResponse<Object> response = ApiResponse.builder()
```

```
            .success(false)
            .message(errorMessage)
            .data(null)
            .build();

    return new ResponseEntity<>(response, HttpStatus.INTERNAL_SERVER_ERROR);
}
```

☑ STEP 9: Add RedisConfig inside config folder

```
package com.advann.product_service.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.cache.RedisCacheManager;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.serializer.GenericJackson2JsonRedisSerializer;
import org.springframework.data.redis.serializer.RedisSerializationContext;
import org.springframework.cache.CacheManager;

import java.time.Duration;

@Configuration
public class RedisConfig {

    @Bean
    public CacheManager cacheManager(RedisConnectionFactory connectionFactory) {

        RedisCacheConfiguration config =
                RedisCacheConfiguration.defaultCacheConfig()
                        .entryTtl(Duration.ofMinutes(10)) // cache expiry
                        .serializeValuesWith(
                                RedisSerializationContext.SerializationPair
                                        .fromSerializer(new GenericJackson2JsonRedisSerializer())
                        );

        return RedisCacheManager.builder(connectionFactory)
                .cacheDefaults(config)
                .transactionAware()
                .build();
    }
}
```

Now how will you check that Redis is working or not?

Run the docker services. Then in another terminal run this command

docker exec -it redis redis-cli monitor

After running this command, you will see OK

Then after that when you will hit the api (getProductById) you will see:

1772471004.486017 [0 172.18.0.5:52976] "GET" "products::1"

1772471005.732423 [0 172.18.0.5:52976] "GET" "products::1"

1772471189.213897 [0 172.18.0.5:52976] "GET" "products::1"